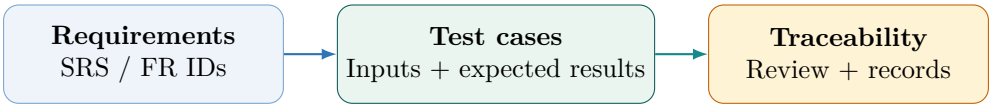
Functional Test Case Design

kvstore

Distributed, Transactional and Replicated Key–Value Store



Experiment aim
Design functional test cases for selected `kvstore` requirements, specify repeatable inputs and observable expected results, include positive and negative cases, and trace every test to a requirement. The submission follows the nine sections of the Lab 9 Student Submission Template.

Student name	Neha Nupur
Enrollment / roll no.	20251651064
Course / batch	MCA – Software Engineering, 3rd Semester
Subject code	CS545
Academic session	2026
Date	24 October 2026
Group No.	1

Team Members

Neha Nupur (Team Leader)	20251651064
Aditi Garg (Member)	20251651008
Shivam Kumar Soni (Member)	20251651084

Contents

1 Aim **2**

2 Materials and Reference **2**

3 List the Requirements to Test **2**

3.1 Quick lookup: what each cited SRS requirement means 2

4 Test Data and Assumptions **4**

5 Test Case Register **4**

6 Requirement Traceability Check **5**

7 Detailed Test Case Records **5**

7.1 Test case detail 1 — TC-01 6

7.2 Test case detail 2 — TC-06 6

8 Review Checklist **7**

9 Result / Learning **7**

1 Aim

To design functional test cases for selected requirements of the `kvstore` distributed key–value store, define clear inputs and preconditions, state observable expected results, include both positive and negative cases, and link each test case to its requirement ID.

2 Materials and Reference

- The `kvstore` project SRS and project documentation.
- The Lab 9 Student Submission Template and Functional Test Case Design instructions.
- The supplied Go source archive, including the HTTP API, storage engine, Raft replication, MVCC transaction logic, WAL recovery, metrics, TLS, operational query subsystem, and shard gateway.

Execution-status convention. The repository contains automated tests for the two detailed scenarios, and the project README reports that the full Go test suite passed in a previous run. The detailed records therefore distinguish that prior automated evidence from a fresh rerun: the current environment has Go 1.23.2, while the project declares Go 1.25.0, so the tests could not be rerun here. The status below is explicitly labelled as a prior-suite report, not a claim of fresh execution in this update.

3 List the Requirements to Test

The template asks for at least three functional requirements. Four project-specific requirement groups are used below. The SRS references have been checked against the supplied `kvstore` SRS and corrected to use its actual three-digit requirement numbering.

Req. ID	Functional requirement in simple words	SRS reference
R-01	The system shall create/update, retrieve, and delete key–value entries through the transactional storage interface and REST API; a missing key must be reported as absent rather than returning an invented value.	FR-032, FR-034, FR-066
R-02	The system shall provide snapshot-isolated transactions and reject conflicting concurrent writes using first-committer-wins conflict detection.	FR-023–FR-028, FR-032–FR-033
R-03	The system shall durably record Raft log data, recover committed operations after restart, commit replicated writes only after durable majority agreement, and help clients find or redirect to the leader.	FR-007–FR-010, FR-015, FR-037, FR-040–FR-042
R-04	The system shall provide REST and operational metrics, safely handle natural-language operational queries, validate TLS certificates, and enforce the ordinary-versus-dedicated cross-shard transaction boundary.	FR-038–FR-039, FR-043–FR-048, FR-058–FR-061, FR-063–FR-066, FR-072–FR-079

3.1 Quick lookup: what each cited SRS requirement means

Use this table to check the meaning of every functional requirement ID cited above without searching through the full SRS. The descriptions are concise paraphrases of the corresponding requirements in Section 4 of the supplied SRS.

Numbering note. In the SRS, the Hybrid Logical Clock requirements are FR-001–FR-006, so FR-004, FR-005, and FR-006 are about clock behavior, not basic key–value CRUD. The table above therefore maps CRUD to the SRS requirements that actually describe the storage API and REST API.

SRS ID	What the requirement specifies
FR-007	Append each write-ahead-log (WAL) record with a checksum.
FR-008	Physically flush each appended WAL record before reporting it as durably persisted.
FR-009	During replay, stop at the first truncated or checksum-invalid record and do not trust records after it.
FR-010	Use the WAL to persist Raft log entries, current term/vote information, and periodic snapshots.
FR-015	Replicate accepted operations and mark them committed only after a durable majority, including the leader, has stored them.
FR-023	Keep an ordered version history for each key, with each version tagged by its HLC timestamp.
FR-024	Assign each transaction a start timestamp.
FR-025	Give each transaction snapshot-consistent reads as of its start timestamp.
FR-026	Reject a transaction at commit if a key it writes has a newer committed version; this is first-committer-wins.
FR-027	Perform the authoritative transaction-conflict check deterministically when applying the transaction from the Raft log on each replica.
FR-028	Perform an early, optimistic conflict check before proposing a transaction through Raft.
FR-032	Expose a transactional API with begin, get, put, delete, and commit operations.
FR-033	Submit a transaction's complete write set as one Raft log entry and wait for safe majority application before reporting success.
FR-034	Apply committed Raft log entries in order to the actual key–value state on every node.
FR-037	When a request reaches a non-leader, return a not-leader response and include the leader address when known.
FR-038	Support optional TLS encryption for connections.
FR-039	Support mutual TLS, requiring client certificates signed by a configured trusted authority.
FR-040	Provide a client library that can discover the current cluster leader.
FR-041	Automatically redirect a request to the leader when a non-leader response includes a leader hint.
FR-042	Retry requests across candidate cluster addresses when one or more addresses are temporarily unreachable.
FR-043	Provide counters, gauges, and histograms as instrumentation primitives.
FR-044	Calculate histogram count, sum, minimum, maximum, running mean, and running standard deviation.
FR-045	Calculate an exponentially weighted moving average for each histogram.
FR-046	Estimate histogram percentiles using fixed logarithmic buckets.
FR-047	Measure latency and success/failure outcomes for each client request, grouped by operation.
FR-048	Expose metrics in Prometheus text format and plain JSON.
FR-058	Answer supported common operational questions using a deterministic rule-based natural-language parser.
FR-059	If configured, use an LLM fallback only to select from a fixed, validated set of read-only operations and verify the selection before execution.
FR-060	Optionally provide already-computed anomaly evidence to an LLM for asynchronous, rate-limited, prose-only root-cause hypotheses; it must not execute recommendations or block monitoring.
FR-061	Continue to provide deterministic insights when the optional LLM is unavailable, unconfigured, or fails.
FR-063	Generate a temporary self-signed certificate for local use when TLS is enabled without an operator-supplied certificate/key.
FR-064	Load an operator-supplied certificate and private key for deployment.
FR-065	Support mutual TLS client-certificate verification against a configured trusted authority.
FR-066	Provide a REST-style API for single-key read/write/delete, multi-key transactions, key browsing, node status, metrics, and operational-intelligence capabilities.
FR-072	Reject an ordinary multi-key transaction before forwarding if its keys span more than one shard.
FR-073	Provide a dedicated cross-shard endpoint that coordinates atomic writes/deletes across involved shards using two-phase commit.
FR-074	Durably record the participating shards before contacting any shard for a cross-shard transaction.
FR-075	Prepare each involved shard by locking affected keys; abort if any shard refuses to prepare.

SRS ID	What the requirement specifies
FR-076	Durably record the final commit decision before instructing any shard to act on it.
FR-077	Send the recorded commit/abort decision to all involved shards and retry transient failures until completion is confirmed.
FR-078	On gateway restart, replay the durable decision log and abort transactions without a durably recorded final decision.
FR-079	Reject cross-shard transactions when the gateway has no configured durable data directory.

4 Test Data and Assumptions

The following data and assumptions are used to make the cases repeatable. They describe test setup and project behavior, not additional product requirements.

- **Test namespace:** use keys such as `lab9/course`, `lab9/a`, and `lab9/b`; confirm or reset their initial state before each run.
- **Basic key operations:** use a running cluster with an elected leader and a client that can reach at least one node.
- **Transactions:** for a same-shard transaction, both keys must map to the same shard. For a cross-shard case, verify that the keys map to different shards before sending the request.
- **Concurrency:** begin two transactions from snapshots that include the same existing version, then attempt to commit writes to the same key.
- **Recovery:** confirm a write has committed before stopping/restarting a node; do not treat an uncommitted proposal as durable.
- **TLS:** run the endpoint with TLS enabled and configure a client that does not trust the presented server certificate. Do not disable certificate verification to make the test pass.
- **API and metrics:** enable the HTTP API and use the routes documented by the project build. The metrics endpoints include `/api/v1/metrics` and `/api/v1/metrics.json`; the dedicated cross-shard transaction endpoint is `/api/v1/xtxn`.
- **Status recording:** write the expected result before execution. Record Pass or Fail only after observing the actual result; otherwise retain Not run.

5 Test Case Register

The register contains 13 cases, including positive and negative scenarios. Expected results are stated before execution. “Positive” means a valid operation or expected successful system behavior; “Negative” means invalid input or a condition that should be safely rejected.

Test ID	Req.	Scenario	Input / precondition	Expected result	Type
TC-01	R-01	PUT then GET a new key.	Absent key `lab9/course`; value `MCA`.	PUT succeeds; GET returns exactly `MCA`.	Positive
TC-02	R-01	GET a missing key.	Unique key that has not been created.	System returns not-found / absent state and does not invent a value.	Negative
TC-03	R-01	DELETE then GET.	Create `lab9/course`, then delete it.	DELETE succeeds; later GET reports the key as absent.	Positive
TC-04	R-01	Delete an already absent key.	Unique missing key.	System returns its defined missing-key result; cluster remains healthy and no unrelated key changes.	Negative

Test ID	Req.	Scenario	Input / precondition	Expected result	Type
TC-05	R-02	Commit two writes in one transaction.	Two keys mapped to the same shard.	Commit succeeds and both values are visible after commit.	Positive
TC-06	R-02	Conflicting concurrent writes.	Two transactions share a snapshot and write the same key.	First valid commit wins; the conflicting commit aborts/rejects and does not overwrite the committed version.	Negative
TC-07	R-03	Restart after a committed write.	Write committed and confirmed before node restart.	After recovery, the committed key/value remains available.	Positive
TC-08	R-03	Send a request through a follower.	Multi-node cluster; client initially contacts a non-leader.	Client redirects/retries to the leader and receives the correct operation result.	Positive
TC-09	R-04	Request Prometheus metrics.	HTTP API and metrics instrumentation enabled.	Metrics route returns a response containing metric samples in Prometheus text format.	Positive
TC-10	R-04	Submit an unrecognized natural-language query.	Query not supported by the rule-based intent parser.	Safe help/fallback response is returned; no arbitrary command is executed.	Negative
TC-11	R-04	Connect with an untrusted TLS certificate.	TLS enabled; client does not trust server certificate.	TLS handshake/connection is rejected; certificate verification is not bypassed.	Negative
TC-12	R-04	Send a valid REST PUT request.	HTTP API enabled; valid request body and key/value.	API returns its documented success response; subsequent read returns the submitted value.	Positive
TC-13	R-04	Cross-shard transaction behavior.	Two keys map to different shards; compare dedicated `/api/v1/xtxn` with ordinary `/api/v1/txn`.	Dedicated endpoint coordinates the cross-shard operation; ordinary endpoint rejects cross-shard transaction without partial mutation.	Positive / Negative

6 Requirement Traceability Check

"Yes" means the register contains at least one positive or negative scenario for the requirement group. TC-13 intentionally contains both a positive dedicated-endpoint scenario and a negative ordinary-endpoint scenario.

Req. ID	Related test IDs	Positive?	Negative?
R-01	TC-01, TC-02, TC-03, TC-04	Yes	Yes
R-02	TC-05, TC-06	Yes	Yes
R-03	TC-07, TC-08	Yes	No
R-04	TC-09, TC-10, TC-11, TC-12, TC-13	Yes	Yes

Traceability check: Every test case has a unique TC identifier and maps to at least one requirement group. Both valid and invalid cases are represented. Requirement groups map to the original SRS identifiers in Section 3.

7 Detailed Test Case Records

The template asks for complete records for two test cases. These records include preconditions, input data, repeatable steps, expected results, actual-result space, and execution status.

7.1 Test case detail 1 — TC-01

Test case ID / requirement ID	TC-01 / R-01 (SRS: FR-032, FR-066)
Test case title and type	Create a key and read it back / Positive
Preconditions	Cluster is running; a leader is elected; client can reach the cluster; the test key is absent or unique.
Test input / test data	Key: `lab9/course`; value: `MCA`; operation sequence: PUT then GET.
Steps to follow	1. Confirm the cluster is healthy. 2. PUT the key `lab9/course` with value `MCA`. 3. Record the write response and confirm it reports success/commit. 4. GET the same key using the supported client/API. 5. Compare the returned value with the submitted value.
Expected result	PUT is successful and GET returns exactly `MCA`; no unrelated key is modified.
Actual result / evidence	Prior automated evidence: `TestHTTPKVRoundTrip` covers the HTTP PUT/GET round trip, and the repository README reports the suite passed previously. A fresh rerun was blocked here because Go 1.23.2 is installed while `go.mod` requires Go 1.25.0.
Execution status	**Pass (prior suite report)**

Pass/fail rule: Pass only if the write is committed and the subsequent GET returns exactly `MCA`. Mark Fail if the returned value differs or the key is missing after a reported successful write. The status above reflects prior repository test-suite evidence; rerun the case on the submission build if fresh execution evidence is required.

7.2 Test case detail 2 — TC-06

Test case ID / requirement ID	TC-06 / R-02 (SRS: FR-023, FR-026, FR-027)
Test case title and type	Concurrent conflicting writes / Negative
Preconditions	Cluster is healthy; key `lab9/shared` has a committed initial value; two transactions can begin from snapshots containing that version.
Test input / test data	Transaction A writes `lab9/shared = A`; transaction B writes `lab9/shared = B`; both begin before either commits.
Steps to follow	1. Store an initial value for `lab9/shared`. 2. Begin transactions A and B so both read the same committed version. 3. Stage a different value for the same key in each transaction. 4. Commit transaction A and record its result. 5. Attempt to commit transaction B and inspect the result and final key value.
Expected result	The first valid commit succeeds; the later conflicting commit is rejected/aborted under first-committer-wins; the final value is A and no partial conflicting write is applied.
Actual result / evidence	Prior automated evidence: `TestSnapshotIsolationWriteConflictFirstCommitterWins` covers the first-committer-wins conflict scenario, and the repository README reports the suite passed previously. A fresh rerun was blocked here because Go 1.23.2 is installed while `go.mod` requires Go 1.25.0.
Execution status	**Pass (prior suite report)**

Pass/fail rule: Pass if one transaction commits and the conflicting transaction is rejected without overwriting the committed value. Mark Fail if both conflicting writes commit as though no conflict existed or the final state is inconsistent. The status above reflects prior repository test-suite evidence; rerun the case on the submission

build if fresh execution evidence is required.

8 Review Checklist

☐ Every test has a unique ID.	☐ Every test maps to a requirement.
☐ Both valid and invalid cases are included.	☐ Expected results are clear.
☐ Test steps can be repeated by another person.	☐ Rules match the project SRS / documented behavior.

Reviewer note: Tick each item after checking the final submission and, where required, after executing the relevant test cases.

9 Result / Learning

In this lab, I designed **13 functional test cases** for the **`kvstore` distributed transactional key–value store**.

I learned how to derive testable scenarios from functional requirements, distinguish positive and negative tests, specify test inputs and observable expected results, document repeatable test steps, and maintain requirement-to-test traceability. I also learned to distinguish prior automated-suite evidence from a fresh execution on the current build.